



MOBILNE APLIKACIJE

VEŽBE 6

Dobavljači sadržaja

2025/2026

Sadržaj

1. Deljena podešavanja	3
2. SQLite	8
4. Dobavljači sadržaja	12
4.1 Jedinствena identifikacija resursa	12
4.2 Aplikacioni dobavljači sadržaja	12
4.2.1 Pristupanje ContentProvider-u druge aplikacije	16
5. Firebase	18
5.1. Realtime Database	18
5.2. Cloud Firestore	19
5.2.1. Implementacija	19
5.3. Firebase Assistant	22
6. Domaći	23

1. Deljena podešavanja

Deljena podešavanja (*SharedPreferences*) olakšavaju perzistentno skladištenje prostih tipova podataka. Podatke čuvamo u datoteci kao uređene parove (ključ, vrednost).

Kreirali smo klasu *SettingsFragment* koja nasleđuje *PreferenceFragmentCompat*. Preporučuje se da podešavanja unutar aplikacija budu kreirana putem ovog fragmenta jer omogućuje rad sa ugrađenim metodama. Na slici 1 prikazan je deo implementacije ovog fragmenta. Unutar metode *onCreatePreferences()* postavljamo naziv našeg fajla sa podešavanjima kako bismo mu kasnije mogli putem tog naziva pristupiti.

```
public class SettingsFragment extends PreferenceFragmentCompat {  
    2 usages  
    private BroadcastReceiver receiver;  
    4 usages  
    private PreferenceManager prefMgr;  
  
    JelenaMatkovic  
    private static SettingsFragment newInstance() {  
        Bundle args = new Bundle();  
  
        SettingsFragment fragment = new SettingsFragment();  
        fragment.setArguments(args);  
        return fragment;  
    }  
  
    1 usage JelenaMatkovic  
    @Override  
    public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {  
        prefMgr = getPreferenceManager();  
        prefMgr.setSharedPreferencesName("pref_file");  
        prefMgr.setSharedPreferencesMode(MODE_PRIVATE);  
        addPreferencesFromResource(R.xml.preferences);  
    }  
}
```

Slika 1. *PrefsFragment*

Ovaj fragment neće imati svoj *layout*, već ćemo koristiti posebnu specifikaciju, koja opisuje koja podešavanja imamo. Slika 2 prikazuje isečak *preferences.xml* fajla u kom se definiše izgled. U elementu *<PreferenceScreen>* se definišu samo elementi koji su sastavni deo podešavanja (nema *layout*-a).

```

<?xml version="1.0" encoding="utf-8"?>
<PreferenceScreen xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <PreferenceCategory
        android:title="Auto data sync"
        android:icon="@drawable/ic_action_settings"
        android:key="sync_settings">

        <CheckBoxPreference
            android:defaultValue="false"
            android:key="pref_sync"
            android:summary="Allow app to sync data automatically, when wifi is ava..."
            android:title="Allow Sync" />

        <ListPreference
            android:dependency="pref_sync"
            android:dialogTitle="Sync interval"
            android:entries="@array/pref_syncConnectionTypes_entries"
            android:entryValues="@array/pref_syncConnectionTypes_values"
            android:key="pref_sync_list"
            android:title="Sync interval"
            android:summary="%s"
            android:defaultValue="1"/>

    </PreferenceCategory>

```

Slika 2. *preferences.xml*

PreferenceCategory

Ovaj element služi za grupisanje drugih elemenata. Grupe su u podešavanjima malo odvojene, razdvojene linijom. Atributi koje navodimo su: naslov i ključ grupe.

CheckBoxPreference

Ovaj element može da prikazuje dva stanja (polje je označeno ili nije).

Atributi koje navodimo su:

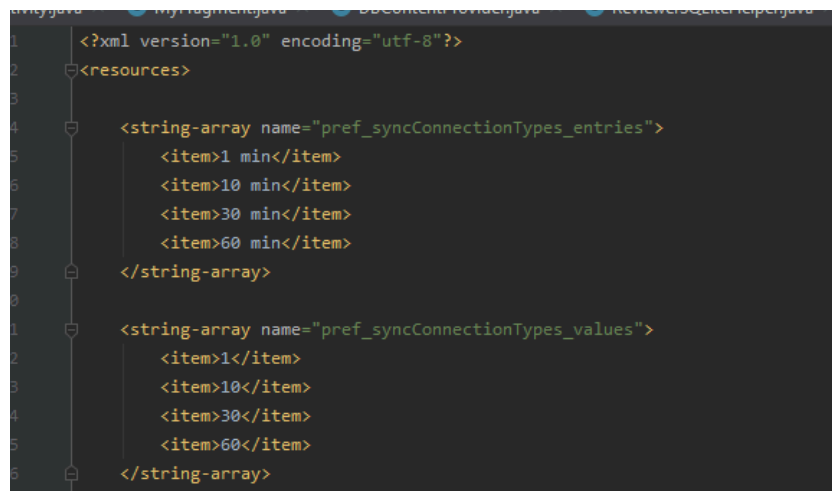
- *defaultValue* – kada pokrenemo aplikaciju ovo će biti podrazumevana vrednost, u našem primeru vrednost je *false*
- *key* - ključ
- *summary* – kratak opis
- *title* – naziv

ListPreference

Pre elementa *ListPreference* uglavnom prethodi *checkBox* element za koji vezujemo listu. Bitni atributi koje navodimo za listu su:

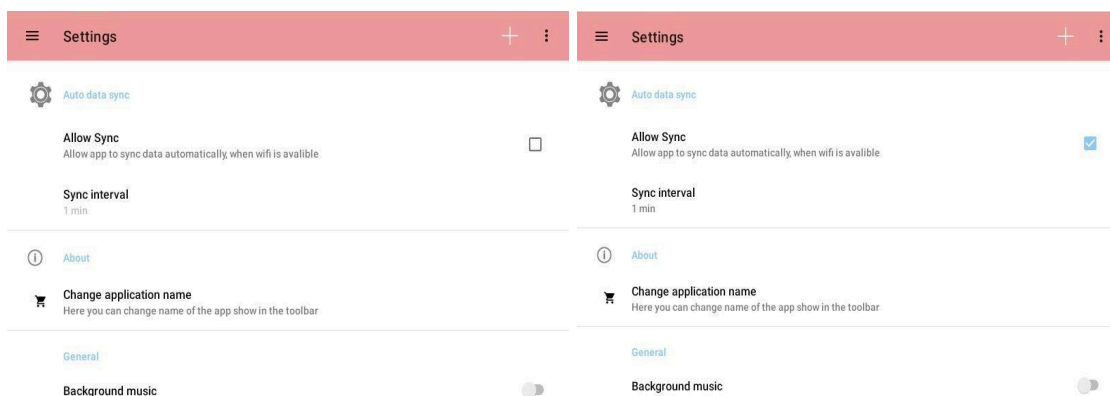
- *dependency* – na ovaj atribut postavljamo ključ *checkBoxPreference* elementa za koji želimo da vežemo ovu listu. Ako korisnik ne čekira prethodno definisan *checkBoxPreference*, onda će lista biti onemogućena za manipulaciju. Primer se vidi na slici 4. Na slici levo *checkBox* nije označen, a desno jeste.
- *summary* - navodimo kog tipa će biti elementi te liste koje prikazujemo (%s-string).
- *entries* - govori šta će korisniku biti prikazano kad se otvori lista.
- *entryValues* - su vrednosti koje će biti upisane u *sharedPreferences* za odgovarajući element.

Sledi da liste *entries* i *entryValues* moraju biti iste dužine (za svaki *entry* koji se prikazuje, postoji odgovarajući *entryValue*, koji se postavlja kao vrednost). Za prvu listu smo postavili listu *pref_syncConnectionTypes_entries*, a za drugu *pref_syncConnectionTypes_values*. Te dve liste smo definisali u datoteci *arrays.xml* unutar direktorijuma *res/values/* (slika 3). Na slici 5 se vidi kako to izgleda na primeru, kada korisnik klikne na *Sync interval*.

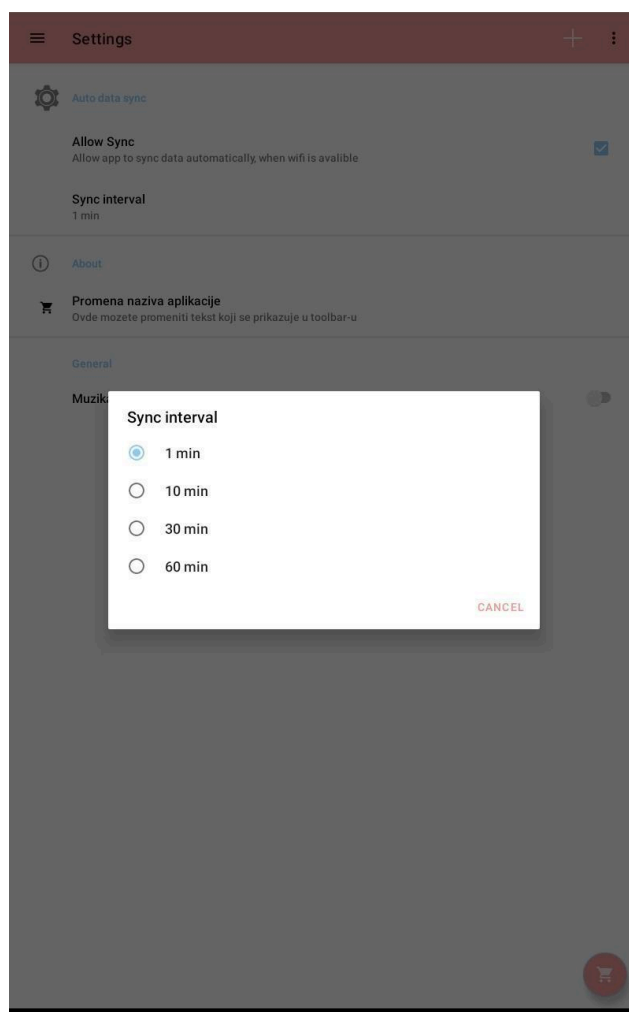


```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3
4     <string-array name="pref_syncConnectionTypes_entries">
5         <item>1 min</item>
6         <item>10 min</item>
7         <item>30 min</item>
8         <item>60 min</item>
9     </string-array>
10
11     <string-array name="pref_syncConnectionTypes_values">
12         <item>1</item>
13         <item>10</item>
14         <item>30</item>
15         <item>60</item>
16     </string-array>
```

Slika 3. Primeri nizova koji se prikazuju



Slika 4. Prikaz podešavanja



Slika 5. Prikaz liste sa opcijama koja se prikazuje kada se klikne na *Sync interval*

U *HomeActivity* klasi se nalazi metoda *consultPreferences()*. U ovoj metodi pristupamo deljenim podešavanjima tako što pozivamo metodu *getSharedPreferences* ugrađene klase *Context* (slika 6).

Metoda *contains* proverava da li postoji podesavanje pod prosleđenim naslovom u okviru Shared Preferences. Metoda *getString* vraća string iz podešavanja, a metoda *getBoolean* vraća boolean vrednost. Gore smo napomenuli da u deljena podešavanja smeštamo parove (ključ, vrednost), te ćemo ključeve koristiti da dobavimo određene vrednosti. Kada pozivamo metode za dobavljanje, prosleđujemo 2 parametra:

- ključ - prvi parametar je ključ, čiju vrednost želimo da dobavimo.
- podrazumevana vrednost - drugi parametar je vrednost, koja treba da nam se vrati nazad, u slučaju da ništa nije zapisano pod prosleđenim ključem.

```
135 private void consultPreferences(){
136
137     /*...*/
138     /*...*/
139     sharedPreferences = getSharedPreferences("com.example.yezbeo_preferences", Context.MODE_PRIVATE);
140     /*...*/
141     if(sharedPreferences.contains("pref_sync_list")){
142         Log.i( tag: "REZ_SP", msg: "Postoji pref_sync_list ključ");
143         /*...*/
144         syncTime = sharedPreferences.getString( s: "pref_sync_list", s1: "1"); // pola minuta
145         Log.i( tag: "REZ_SP", msg: "SYNC TIME = " + syncTime.toString());
146     }
147
148     if(sharedPreferences.contains("pref_sync")){
149         Log.i( tag: "REZ_SP", msg: "Postoji pref_sync ključ");
150         allowSync = sharedPreferences.getBoolean( s: "pref_sync", b: false);
151         Log.i( tag: "REZ_SP", msg: "ALLOW SYNC = " + allowSync);
152     }
153
154     if(sharedPreferences.contains("pref_name")){
155         Log.i( tag: "REZ_SP", msg: "Postoji pref_name ključ");
156         String name = sharedPreferences.getString( s: "pref_name", s1: "default");
157         Log.i( tag: "REZ_SP", msg: "PREF NAME = " + name);
158     }else{
159         Log.i( tag: "REZ_SP", msg: "Ne postoji pref_name ključ");
160     }
161 }
```

Slika 6. Čitanje zapisanih vrednosti

Na sličan način možemo da pristupimo deljenim podešavanjima i da smestimo neko podešavanje unutar njega. Pomoću metode *putString*, *putBoolean* itd. možemo da dodamo podešavanje sa nekim ključem i vrednošću (slika 7).

```
private void setAppName(){
    SharedPreferences.Editor sp_editor = sharedPreferences.edit();
    Log.i( tag: "REZ_SP", msg: "Set pref_name ključ");
    if(sharedPreferences.contains("pref_name")){
        String s = sharedPreferences.getString( s: "pref_name", s1: "ShopApp");
        sp_editor.putString( s: "pref_name", s);
        sp_editor.commit();
        getSupportActionBar().setTitle(s);
    }
}
```

Slika 7. Zapisivanje vrednosti u deljenim podešavanjima

2. SQLite

Android aplikacije mogu da koriste ugrađen sistem za upravljanje bazama podataka (*SQLite*). *SQLite* se izvršava u istom procesu kao i sama aplikacija.

Baza podataka predstavljena je klasom *SQLiteDatabase*.

CRUD operacije nad bazom podataka izvršavaju se pozivom *insert*, *query*, *update* i *delete* metoda.

query metoda je specifična jer kao rezultat vraća *Cursor*. *Cursor* vraća adresu gde se podaci nalaze tj. relacija koja je rezultat SQL upita predstavljena je kursorom. Kursore koristimo da bismo vršili navigaciju kroz rezultat upita.

U bazi svaka tabela ima redove i kolone. Potrebno je pozicionirati se na odgovarajući red i odgovarajuću kolonu. Prvo se pozicioniramo na red sa nekom od metoda:

- `boolean move(int offset)`

Pomeri se za određeni broj redova od trenutne pozicije.

- `boolean moveToFirst()`

Pređi na prvi red.

- `boolean moveToLast()`

Pređi na poslednji red

- `boolean moveToNext()`

Pređi na sledeći red.

- `boolean moveToPrevious()`

Pređi na prethodni red.

Kada se pozicioniramo na red, sledeći korak je da odaberemo kolonu, tj. da pročitamo rezultat upita. Za to možemo da iskoristimo neku od metoda:

- `int getCount()`
- `int getColumnIndex(String column_name)`
- `String getColumnName(int column_index)`
- `String getString(int column_index)`
- `int getInt(int column_index)`
- `long getLong(int column_index)`
- `float getFloat(int column_index)`

U primeru za ove vežbe kreirali smo klasu *SQLiteHelper* koja nasleđuje klasu *SQLiteOpenHelper*. Klasa *SQLiteOpenHelper* nam omogućava da napravimo, izmenimo ili otvorimo bazu podataka.

Implementiramo metode:

- `void onCreate(SQLiteDatabase database)`
- `void onOpen(SQLiteDatabase database)`

- `void onUpgrade(SQLiteDatabase database, int oldVersion, int newVersion)`
- `void onDowngrade(SQLiteDatabase database, int oldVersion, int newVersion)`

onCreate

U ovoj metodi kreiramo našu bazu tako što pozivamo `db.execSQL` za kreiranu tabelu (slika 8).

```
@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(DB_CREATE);
}
```

Slika 8. Kreiranje tabele

onUpgrade

Metodu `onUpgrade` pozivamo kad baza treba da se promeni (slika 9). Prvi korak je da dropujemo sve tabele koje imamo.

```
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    Log.i( tag: "REZ_DB", msg: "ON UPGRADE SQLITE HELPER");
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_PRODUCTS);
    onCreate(db);
}
```

Slika 9. Izmena tabele

U klasi `Util` kreirali smo metodu `initDB` koja služi za inicijalizaciju naše baze (slika 10). U toj metodi smo kreirali dva entiteta i uneli ih u tabelu pozivanjem metode `insert`. Metoda se može pozvati unutar `HomeActivity`-a kako aplikacija ne bi imala praznu listu proizvoda prilikom pokretanja.

```

public class Util {
    public static void initDB(Activity activity) {
        SQLiteHelper dbHelper = new SQLiteHelper(activity);
        SQLiteDatabase db = dbHelper.getWritableDatabase();
        Log.i( tag: "REZ_DB", msg: "ENTRY INSERT TO DATABASE");
        {
            ContentValues entry = new ContentValues();
            entry.put(SQLiteHelper.COLUMN_TITLE, "Iphone");
            entry.put(SQLiteHelper.COLUMN_DESCRIPTION, "15 pro");
            entry.put(SQLiteHelper.COLUMN_IMAGE, -1);

            activity.getContentResolver().insert(DBContentProvider.CONTENT_URI_PRODUCTS, entry);

            entry = new ContentValues();
            entry.put(SQLiteHelper.COLUMN_TITLE, "Samsung");
            entry.put(SQLiteHelper.COLUMN_DESCRIPTION, "S23 ultra");
            entry.put(SQLiteHelper.COLUMN_IMAGE, -1);

            activity.getContentResolver().insert(DBContentProvider.CONTENT_URI_PRODUCTS, entry);
        }

        db.close();
    }
}

```

Slika 10. Metoda za inicijalizacije *SQLite* baze

Na slici 11 prikazano je definisanje naziva tabele i kolona za model proizvoda unutar klase *SQLiteHelper* koji su nam neophodni za rad sa podacima iz baze.

```

10 usages
public static final String TABLE_PRODUCTS = "PRODUCTS";
7 usages
public static final String COLUMN_ID = "_id";
5 usages
public static final String COLUMN_TITLE = "title";
5 usages
public static final String COLUMN_DESCRIPTION = "description";
3 usages
public static final String COLUMN_IMAGE = "image";

//Dajemo ime bazi
1 usage
private static final String DATABASE_NAME = "products.db";
//i pocetnu verziju baze. Obicno krece od 1
1 usage
private static final int DATABASE_VERSION = 1;

1 usage
private static final String DB_CREATE = "create table "
    + TABLE_PRODUCTS + "("
    + COLUMN_ID + " integer primary key autoincrement , "
    + COLUMN_TITLE + " text, "
    + COLUMN_DESCRIPTION + " text, "
    + COLUMN_IMAGE + " integer"
    + ")";

```

Slika 11. Definisanje parametara za rad sa tabelom PRODUCTS

4. Dobavljači sadržaja

Dobavljači sadržaja (*ContentProvider*) upravljaju podacima aplikacije i omogućavaju da im se pristupi na standardizovan način. Podaci se mogu nalaziti u bazi podataka, na internetu ili nekom drugom mestu. Npr. postoji dobavljač sadržaja koji dozvoljava korisniku da uređuje listu kontakata na uređaju. Ovo znači da svaka aplikacija, ako ima odobrenje, može da pristupa određenim dobavljačima i da čita ili upisuje neke podatke.

Postoje dve vrste dobavljača sadržaja:

1. Sistemski
2. Aplikacioni

Sistemski dobavljači su uključeni u Android (*Browser, Calendar, CallLog, Contacts..*).

Aplikacione dobavljače sadržaja prave programeri, koji su zaduženi za kreiranje same aplikacije. Ako želimo da resursima naše aplikacije mogu da pristupe neke druge aplikacije, onda moramo to i da im omogućimo kreiranjem dobavljača sadržaja. Kako se prave aplikacioni dobavljači sadržaja predstavljeno je u potpoglavlju 4.2.

4.1 Jedinstvena identifikacija resursa

Resurse opisuje URI i MIME tip.

Primer URI-ja:

`content://user_dictionary/words`

URI se sastoji iz 3 dela:

- šeme,
- imena dobavljača i
- imena tabele.

MIME tip specifikira tip sadržaja (text/plain, text/pdf, image/jpeg, itd.). Kada se navede kao tip npr. *jpeg* to znači da ćemo dobiti samo slike sa tom ekstenzijom.

Primer: U galeriji telefona se čuvaju slike koje kamera napravi. Galeriji pristupamo preko dobavljača sadržaja i tražimo određenu sliku. Kao povratna vrednost stiže URI, a ne slika, jer takva situacija ne bi bila optimalna. Sa tim URI-jem možemo da tražimo direktan pristup resursu.

4.2 Aplikacioni dobavljači sadržaja

Kreiramo klasu *DBContentProvider* koja nasleđuje *ContentProvider*.

Deklarišemo klasu *DBContentProvider* u *AndroidManifest.xml* datoteci (slika 13) tako što navedemo element `<provider>`.

```
<provider
    android:name="com.example.shopapp.database.DBContentProvider"
    android:authorities="com.example.shopapp"
    android:exported="false" />
```

Slika 13. Deklarisanje dobavljača sadržaja u *AndroidManifest* datoteci

Na slici 14 se nalazi primer kreiranja URI-ja. *AUTHORITY* polje je ime dobavljača, što je najčešće id aplikacije. *PRODUCT_PATH* je naziv tabele.

Ako korisnik želi informacije o pojedinačnom resursu onda definišemo *UriMatcher*. Svaki put proveravamo da li je zahtev stigao za celu tabelu ili za pojedinačan red i u zavisnosti od toga se pretražuje baza i vraća se rezultat nazad.

```
private static final int PRODUCTS = 10;
4 usages
private static final int PRODUCT_ID = 20;
3 usages
private static final String AUTHORITY = "com.example.shopapp";
4 usages
private static final String PRODUCT_PATH = "products";
5 usages
public static final Uri CONTENT_URI_PRODUCTS = Uri.parse("content://" + AUTHORITY + "/" + PRODUCT_PATH);
6 usages
private static final UriMatcher sURIMatcher = new UriMatcher(UriMatcher.NO_MATCH);

static {
    sURIMatcher.addURI(AUTHORITY, PRODUCT_PATH, PRODUCTS);
    sURIMatcher.addURI(AUTHORITY, path: PRODUCT_PATH + "/#", PRODUCT_ID);
}
```

Slika 14. Kreiranje URI-ja

Prilikom nasleđivanja *ContentProvider* klase, treba implementirati metode:

- *insert()*,
- *query()*,
- *update()*,
- *delete()*,
- *getType()* i
- *onCreate()*

Na slici 15 se nalazi primer metode *insert*.

```

public Uri insert(Uri uri, ContentValues values) {
    Log.i( tag: "REZ_DB", msg: "INSERT");
    int uriType = sURIMatcher.match(uri);
    SQLiteDatabase sqldb = database.getWritableDatabase();
    long id = 0;
    switch (uriType) {
        case PRODUCTS:
            id = sqldb.insert(SQLiteHelper.TABLE_PRODUCTS, nullColumnHack: null, values);
            break;
        default:
            throw new IllegalArgumentException("Unknown URI: " + uri);
    }
    getContext().getContentResolver().notifyChange(uri, observer: null);
    return Uri.parse( uriString: PRODUCT_PATH + "/" + id);
}

```

Slika 15. Metoda *insert*

Na slici 16 se nalazi primer *query* metode, koja kao rezultat vraća *Cursor*.

```

public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    Log.i( tag: "REZ_DB", msg: "QUERY");
    // Using SQLiteQueryBuilder instead of query() method
    SQLiteQueryBuilder queryBuilder = new SQLiteQueryBuilder();

    // check if the caller has requested a column which does not exist
    //checkColumns(projection);
    int uriType = sURIMatcher.match(uri);
    switch (uriType) {
        case PRODUCT_ID:
            // Adding the ID to the original query
            queryBuilder.appendWhere( inWhere: SQLiteHelper.COLUMN_ID + "=" + uri.getLastPathSegment());
        case PRODUCTS:
            // Set the table
            queryBuilder.setTables(SQLiteHelper.TABLE_PRODUCTS);
            break;
        default:
            throw new IllegalArgumentException("Unknown URI: " + uri);
    }

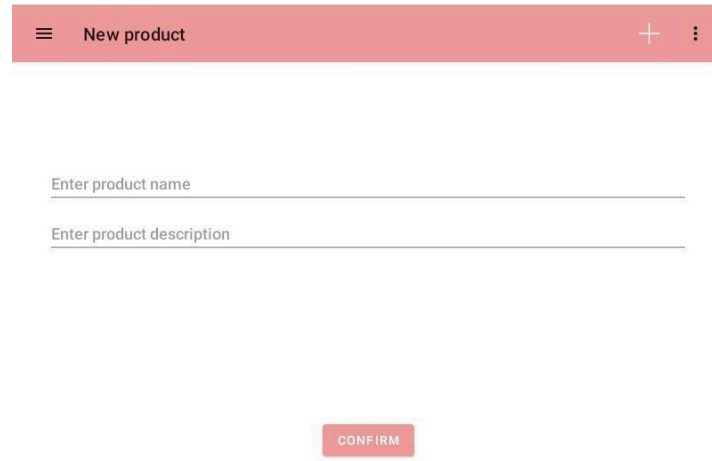
    SQLiteDatabase db = database.getWritableDatabase();
    Cursor cursor = queryBuilder.query(db, projection, selection, selectionArgs, groupBy: null, having: null, sortOrder);
    // make sure that potential listeners are getting notified
    cursor.setNotificationUri(getContext().getContentResolver(), uri);
    return cursor;
}

```

Slika 16. Metoda *query*

U primeru za ove vežbe možete pogledati i sve ostale metode.

Klikom na znak plus unutar *toolbar*-a, otvara se *NewProductFragment* sa poljima za unost naziva i opisa novog proizvoda (slika 17).



Slika 17. Fragment za dodavanje novog proizvoda

Dodavanje proizvoda vrši se metodom *addProduct()* sa slike 18.

```
private void addNewProduct() {
    String title = titleText.getText().toString();
    String description = descrText.getText().toString();
    // only save if either summary or description
    // is available
    if (description.length() == 0 && description.length() == 0) {
        return;
    }
    ContentValues values = new ContentValues();
    values.put(SQLiteHelper.COLUMN_TITLE, title);
    values.put(SQLiteHelper.COLUMN_DESCRIPTION, description);
    Objects.requireNonNull(getActivity()).getContentResolver().insert(
        DBContentProvider.CONTENT_URI_PRODUCTS,
        values);
    getActivity().getSupportFragmentManager().popBackStack();
}
```

Slika 18. Dodavanje novog proizvoda

Šta ako se pojave dve aplikacije koje u isto vreme pošalju zahtev za neki resurs naše aplikacije? Treba da vodimo računa o sinhronizaciji. Sve metode osim metode *onCreate()* treba da budu *thread-safe*. Treba izbegavati izvršavanje dugačkih operacija u metodi *onCreate()*.

Primećujemo da metoda *onDestroy* ne postoji, jer dobavljači sadržaja postoje od početka do kraja procesa.

Klasa ***ContentResolver*** omogućava izvršavanje CRUD (create, read, update, delete) operacija nad skladištem podataka. Podacima pristupamo tako što zatražimo odgovarajuća prava pristupa, a potom izvršavamo upit nad dobavljenim sadržajem.

Upit se postavlja na sličan način na koji se postavlja SQL upit. Sadrži URI, spisak kolona koje treba vratiti (projekciju), uslov koji vraćene vrste treba da zadovolje (selekciju) i način sortiranja rezultata.

Na sličan način na koji je moguće pristupiti podacima, moguće ih je i promeniti.

Primer upotrebe *ContentResolver*-a možete videti na slici 10 kada se vrši inicijalizacija SQLite baze. Metodi *initDB* prosleđujemo aktivnost, u našem primeru se prosleđuje *HomeActivity*, i pozivamo metodu *getContentResolver*, a potom i metodu *insert* da ubacimo kreirani proizvod.

4.2.1 Pristupanje *ContentProvider*-u druge aplikacije

Ukoliko želimo da pristupimo podacima neke aplikacije, to možemo postići obraćanjem *ContentProvider*-u te aplikacije. U nastavku je dat primer preuzimanja naziva kontakata sa korisničkog uređaja. Da bismo pristupili kontaktima, neophodno je da definišemo permisiju unutar *AndroidManifest* fajla (slika 19).

```
<uses-permission android:name="android.permission.READ_CONTACTS"/>
```

Slika 19. Definisane dozvole za pristup čitanju kontakata

Na slici 20 prikazana je metoda *getContacts()* unutar *ProfileFragment*-a koja prikuplja kontakte pomoću *Cursor*-a i *ContentResolver*-a. *Contacts.CommonDataKinds.Phone.CONTENT_URI* predstavlja definisan *URI* aplikacije za kontakte preko koje dobavljamo podatke.

```
private Cursor getContacts() {
    String[] projection = new String[] { ContactsContract.Contacts._ID,
        ContactsContract.Contacts.DISPLAY_NAME };
    Cursor cursor = Objects.requireNonNull(getActivity()).getContentResolver()
        .query(ContactsContract.CommonDataKinds.Phone.CONTENT_URI, projection, selection: null, selectionArgs: null, sortOrder: null);
    if (cursor != null && cursor.moveToFirst()){
        do {
            @SuppressWarnings("Range") String name = cursor.getString(cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME));
        } while (cursor.moveToNext());
    }
    return cursor;
}
```

Slika 20. Dobavljanje kontakata putem *ContentProvider*-a

Pre samog učitavanja kontakata, vršimo proveru dozvole za čitanje kontakata (slika 21). Treba voditi računa da se od novijih verzija Androida ne preporučuje korišćenje metode `onRequestPermissionsResult()` iz fragmenta. Umesto nje, koristimo `registerForActivityResult()` kao što je prikazano na slici 22.

```
private void processContact() {  
    int getContacts = ContextCompat.checkSelfPermission(Objects.requireNonNull(getContext()), permission);  
    if (getContacts != PackageManager.PERMISSION_GRANTED) {  
        mPermissionResult.launch(permission);  
    } else {  
        startProcessingContacts();  
    }  
}
```

Slika 21. Provera permisija za čitanje kontakata

```
mPermissionResult = registerForActivityResult(  
    new ActivityResultContracts.RequestPermission(),  
    result -> {  
        if (result) {  
            startProcessingContacts();  
        } else {  
            Toast.makeText(getActivity(), text: "No permission.", Toast.LENGTH_SHORT).show();  
        }  
    });  
processContact();
```

Slika 22. Definisanje prikaza dijaloga za permisije i rezultata u odnosu na korisnički izbor

Ukoliko korisnik odobri dozvolu, učitani kontakti se prikazuju unutar *ProfileFragment*-a (slika 23).



Slika 23. Prikaz imena učitanih kontakata

5. Firebase

Firebase je platforma za razvoj mobilnih i web aplikacija koju je razvio Google. Ova platforma nudi razne alate i usluge koji omogućavaju programerima da brzo i jednostavno razvijaju i skaliraju svoje aplikacije. Firebase podržava različite vrste aplikacija, uključujući Android, iOS i web aplikacije.

Neke od glavnih usluga koje Firebase nudi su:

- **Realtime Database:** Firebase baza podataka u stvarnom vremenu koja omogućuje aplikacijama da sinhronizuju podatke između korisnika u stvarnom vremenu.
- **Cloud Firestore:** moderna baza podataka koja se temelji na dokumentima, a koristi se za pohranu i sinhronizaciju podataka između korisnika i aplikacija.
- **Authentication:** Omogućava korisnicima da se prijave u aplikaciju putem e-pošte, Facebook-a, Google-a i drugih usluga.
- **Storage:** Firebase omogućuje spremanje datoteka u oblaku, što je korisno za aplikacije koje trebaju spremiti slike, video zapise i druge vrste medija.
- **Cloud Messaging:** Firebase pruža uslugu push obavijesti koja omogućuje aplikacijama da šalju obavijesti na mobilne uređaje.
- **Hosting:** Firebase nudi brz i jednostavan hosting za web aplikacije.

Firebase također nudi i druge usluge poput analitike, testiranja i optimizacije aplikacija, dinamičkih veza, upravljanja korisnicima i još mnogo toga.

5.1. Realtime Database

Realtime Database je NoSQL baza podataka koja je specijalizovana za sinhronizaciju podataka u stvarnom vremenu između korisnika i aplikacija.

Realtime Database koristi JSON format za pohranu i sinhronizaciju podataka. Podaci se pohranjuju u obliku stabla JSON objekata, što omogućuje brzo i jednostavno upravljanje podacima.

Neki od glavnih usluga koje nudi Realtime Database su:

- **Realtime sinhronizacija:** Realtime Database omogućuje automatsku sinhronizaciju podataka u stvarnom vremenu između korisnika i aplikacija. To znači da se promene u podacima automatski ažuriraju na svim uređajima koji koriste aplikaciju.
- **Realtime događaji:** Realtime Database omogućuje praćenje događaja u stvarnom vremenu, poput dodavanja, brisanja i ažuriranja podataka. To je korisno za aplikacije koje zahtevaju praćenje promena u podacima.
- **Offline podrška:** Realtime Database omogućuje rad s podacima i kada nema internetske veze. To je korisno za korisnike koji se nalaze u područjima s lošim pristupom internetu.
- **Sigurnosna pravila:** Realtime Database omogućuje postavljanje pravila sigurnosti za pristup podacima, što pomaže u zaštiti podataka od neovlaštenog pristupa.

- Integracija s drugim Firebase uslugama: Realtime Database se lako integrira s drugim Firebase uslugama kao što su autentifikacija korisnika, upravljanje korisnicima i analitika.

5.2. Cloud Firestore

Cloud Firestore je moderna baza podataka koja se temelji na dokumentima, a koristi se za pohranu i sinhronizaciju podataka između korisnika i aplikacija.

Firestore se temelji na NoSQL modelu podataka, što znači da podaci nisu pohranjeni u relacionim tabelama kao u tradicionalnoj relacionoj bazi podataka. Umesto toga, podaci se pohranjuju u kolekcijama koje sadrže dokumente, a dokumenti mogu sadržavati različite vrste podataka, uključujući tekst, brojeve, datume, objekte i nizove.

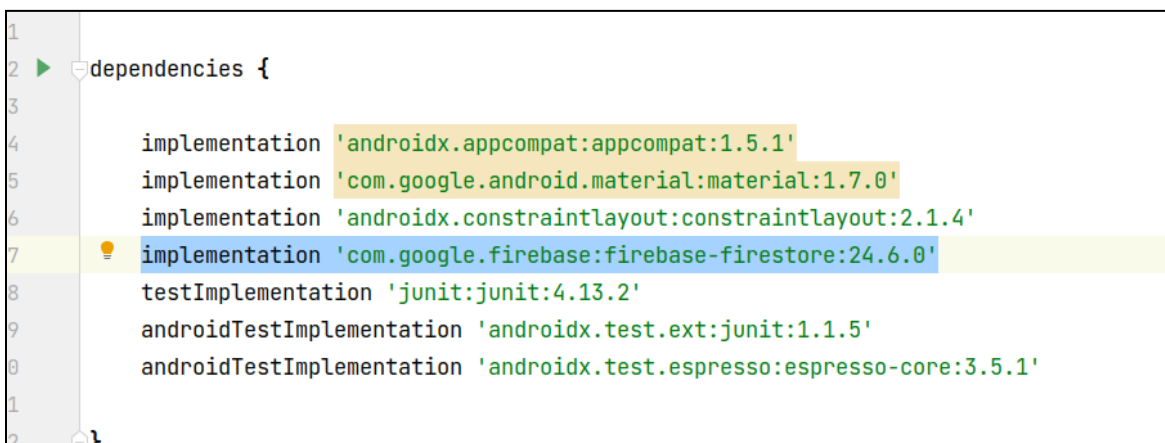
Firestore se odlikuje brzim i skalabilnim pristupom podacima, a koristi se za različite vrste aplikacija, uključujući mobilne i web aplikacije, igre i druge usluge.

Firestore ima i neke druge opcije, uključujući:

- Realtime komunikacija: Firestore omogućuje izvođenje upita u stvarnom vremenu, što znači da se podaci automatski ažuriraju kad se promene.
- Indeksiranje polja: Firestore automatski indeksira polja u dokumentima, što omogućuje brzo pretraživanje podataka i postavljanje upita.
- Sigurnosna pravila: Firestore omogućuje programerima postavljanje pravila sigurnosti za pristup podacima, što pomaže u zaštiti podataka od neovlaštenog pristupa.
- Integracija s drugim Firebase uslugama: Firestore se lako integriše s drugim Firebase uslugama kao što su autentifikacija korisnika, upravljanje korisnicima i analitika.

5.2.1. Implementacija

Da biste koristili Cloud Firestore u vašoj Java aplikaciji, prvo treba da uključite Firebase Firestore biblioteku u vaš projekt (slika 24). Postupak za uključivanje biblioteke zavisi od vašeg razvojnog okruženja, ali obično se radi o dodavanju biblioteke u datoteku **build.gradle** vašeg projekta.



```
1
2 dependencies {
3
4     implementation 'androidx.appcompat:appcompat:1.5.1'
5     implementation 'com.google.android.material:material:1.7.0'
6     implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
7     implementation 'com.google.firebase:firebase-firestore:24.6.0'
8     testImplementation 'junit:junit:4.13.2'
9     androidTestImplementation 'androidx.test.ext:junit:1.1.5'
10    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'
11
12 }
```

Slika 24. Firebase Firestore biblioteka

Nakon uključivanja biblioteke, možete koristiti Firestore klijent za pristup i manipulisanje podacima u vašoj Firestore bazi podataka.

Selektovanje, dodavanje, izmena i brisanje podataka u Firestore bazi podataka implementirane su u *CloudStoreUtil* klasi.

Dodavanje podataka

Dodavanje novog objekta u Firestore bazu podataka dato je na slici 25.

The image shows a code editor with a Java file named `CloudStoreUtil.java`. The code defines a static method `insert()` that creates a new `User` object and adds it to a Firestore collection named "users". The `User` object is initialized with the first name "Milica" and the last name "Ivica". A `HashMap` is used to store these attributes. The code then uses the `FirebaseFirestore` instance to add the user to the "users" collection. It includes success and failure listeners to handle the database operation. The success listener logs the document ID, and the failure listener logs an error message. The code is as follows:

```
public static void insert(){
    // kreiraj novi objekat klase User
    User user1 = new User("Milica", "Ivica");
    Map<String, Object> user1 = new HashMap<>();
    // user.put("firstName", "Ivana");
    // user.put("lastName", "Ivica");

    FirebaseFirestore db = FirebaseFirestore.getInstance();
    // dodaje se novi user u kolekciju "users"
    db.collection(collectionPath: "users") CollectionReference
        .add(user1) Task<DocumentReference>
        .addOnSuccessListener(new OnSuccessListener<DocumentReference>() {
            @Override
            public void onSuccess(DocumentReference documentReference) {
                Log.d( tag: "REZ_DB", msg: "DocumentSnapshot added with ID: " + documentReference.getId());
            }
        })
        .addOnFailureListener(new OnFailureListener() {
            @Override
            public void onFailure(@NonNull Exception e) {
                Log.w( tag: "REZ_DB", msg: "Error adding document", e);
            }
        });
}
```

Slika 25. Dodavanje novog user-a

U ovom primeru, stvaramo novi objekat podataka pomoću klase `HashMap`, a zatim ga dodajemo u kolekciju "users" pomoću Firestore klijenta. Metoda ***addOnSuccessListener*** se poziva kada je user uspešno dodan u bazu podataka, a ***addOnFailureListener*** se poziva ako se dogodi greška prilikom dodavanja user-a.

Dobavljanje podataka

Dobavljanje sadržaja neke tabele iz Firestore baze podataka dato je na slici 26.

```

public static void select(){
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection( collectionPath: "users") CollectionReference
        .get() Task<QuerySnapshot>
        .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>() {
            @Override
            public void onComplete(@NonNull Task<QuerySnapshot> task) {
                if (task.isSuccessful()) {
                    for (QueryDocumentSnapshot document : task.getResult()) {
                        Log.d( tag: "REZ_DB", msg: document.getId() + " => " + document.getData());
                    }
                } else {
                    Log.w( tag: "REZ_DB", msg: "Error getting documents.", task.getException());
                }
            }
        });
}

```

Slika 26. Dobavljanje tabele users

U ovom primeru koristimo metodu **get()** Firestore klijenta kako bismo dobili sve dokumente iz kolekcije "users". Metoda **addOnCompleteListener** se poziva kada se dobave podaci, a zatim se prolazi kroz sve dokumente korišćenjem petlje i ispisuje se ID dokumenta i podaci.

Izmena podataka

Izmena sadržaja nekog objekta iz Firestore baze podataka dato je na slici 27.

```

public static void update(){
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    // izmena dokumenta s ID-em "GFowOuJW63DykTxughJS" iz kolekcije "users"
    DocumentReference docRef = db.collection( collectionPath: "users").document( documentPath: "GFowOuJW63DykTxughJS");
    docRef
        .update( field: "firstName", value: "Dragan")
        .addOnSuccessListener(aVoid -> Log.d( tag: "REZ_DB", msg: "User successfully changed"))
        .addOnFailureListener(e -> Log.w( tag: "REZ_DB", msg: "Error getting documents.", e));
}

```

Slika 27. Izmena objekta iz Firestore baze

U ovom primeru koristimo metodu **update()** Firestore klijenta kako bismo izmenili podatak "firstName" dokumenta s ID-em "GFowOuJW63DykTxughJS" iz kolekcije "users". Metoda **addOnSuccessListener** se poziva kada je dokument uspešno izmenjen, a metoda **addOnFailureListener** se poziva ako se dogodi greška.

Brisanje podataka

Brisanje nekog objekta iz Firestore baze podataka dato je na slici 28.

```

public static void delete(){
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    // izbrisati user-a s ID-om "1wUqKBOWBI501Iq6r0AA" iz kolekcije "users"
    db.collection( collectionPath: "users") CollectionReference
        .document( documentPath: "1wUqKBOWBI501Iq6r0AA") DocumentReference
        .delete() Task<Void>
        .addOnSuccessListener(aVoid -> Log.d( tag: "REZ_DB", msg: "The user has been deleted."))
        .addOnFailureListener(e -> Log.w( tag: "REZ_DB", msg: "Error deleting document.", e));
}

```

Slika 16. Brisanje objekta iz Firestore baze

Ovaj kod koristi metodu **delete** Firestore klijenta za brisanje dokumenta s ID-om "1wUqKBOWBI501Iq6r0AA" iz kolekcije "users". Metoda **addOnSuccessListener** i **addOnFailureListener** se pozivaju kada se brisanje dokumenta završi uspešno ili neuspešno, te ispisuju odgovarajuće poruke u logu.

5.3. Firebase Assistant

Firebase Assistant je alat koji se integriše u Android Studio i omogućava jednostavno dodavanje Firebase funkcionalnosti u aplikaciju. Firebase Assistant pruža vizuelni prikaz Firebase usluga i omogućava brzo dodavanje usluga u aplikaciju.

Kada se Firebase Assistant otvori u Android Studiju (Tools/Firebase opcija), prikazuju se različite usluge koje Firebase nudi, kao što su Realtime Database, Cloud Firestore, autentifikacija korisnika i analitika. Korisnici mogu jednostavno odabrati usluge koje žele koristiti i slediti uputstva za konfiguraciju.

Firebase Assistant također omogućava pristup dokumentaciji i primerima koda za svaku Firebase uslugu, što je korisno za početnike koji se tek upoznaju s Firebase platformom.

Korišćenje Firebase Assistant-a u Android Studio-u olakšava dodavanje Firebase funkcionalnosti u aplikaciju, ubrzava razvoj i pomaže programerima u stvaranju boljih aplikacija.

6. Domaći

Domaći se nalazi na *Canvas-u* (*canvas.ftn.uns.ac.rs*) na putanji *Вежбе/06 Задамак.pdf*.

Primer možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije/>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić (milan.jovkic@uns.ac.rs)
- Jelena Matković (matkovic.jelena@uns.ac.rs)